

Modeling Techniques in VERILOG

Dr. Chandan Karfa

Department of Computer Science and Engineering

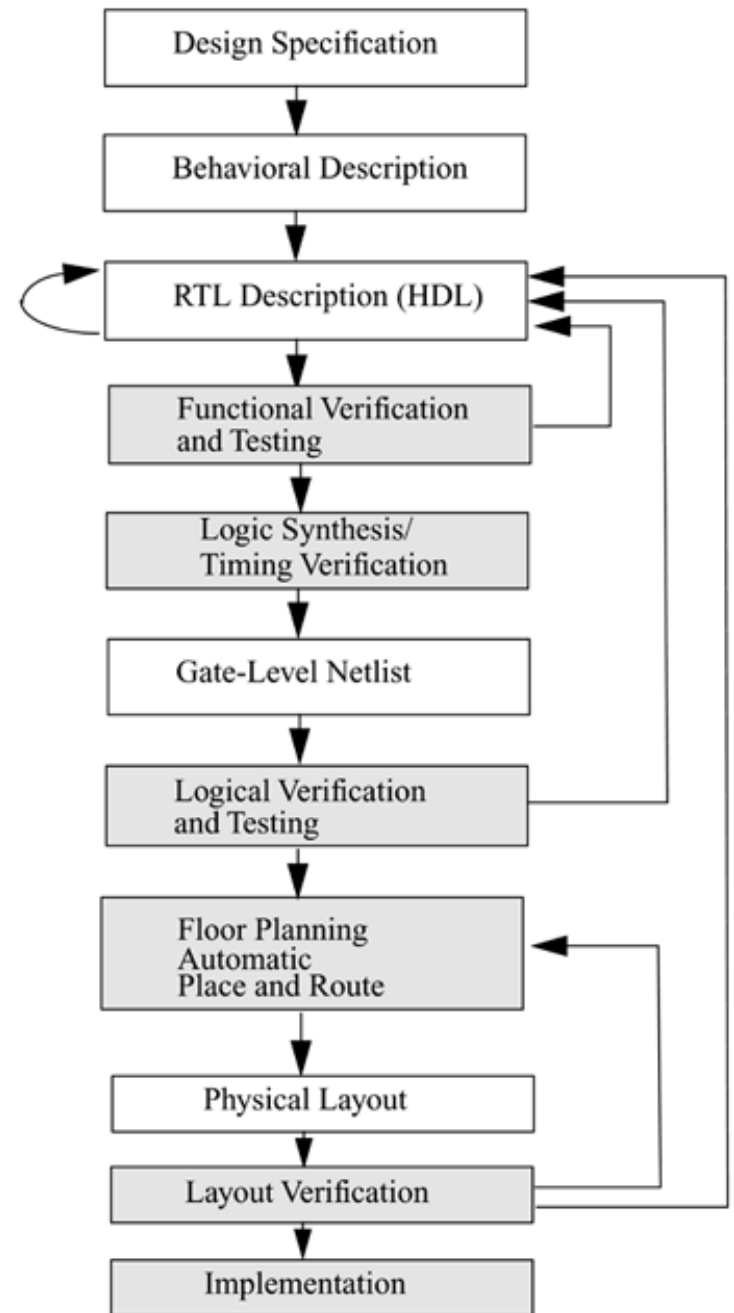


भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI



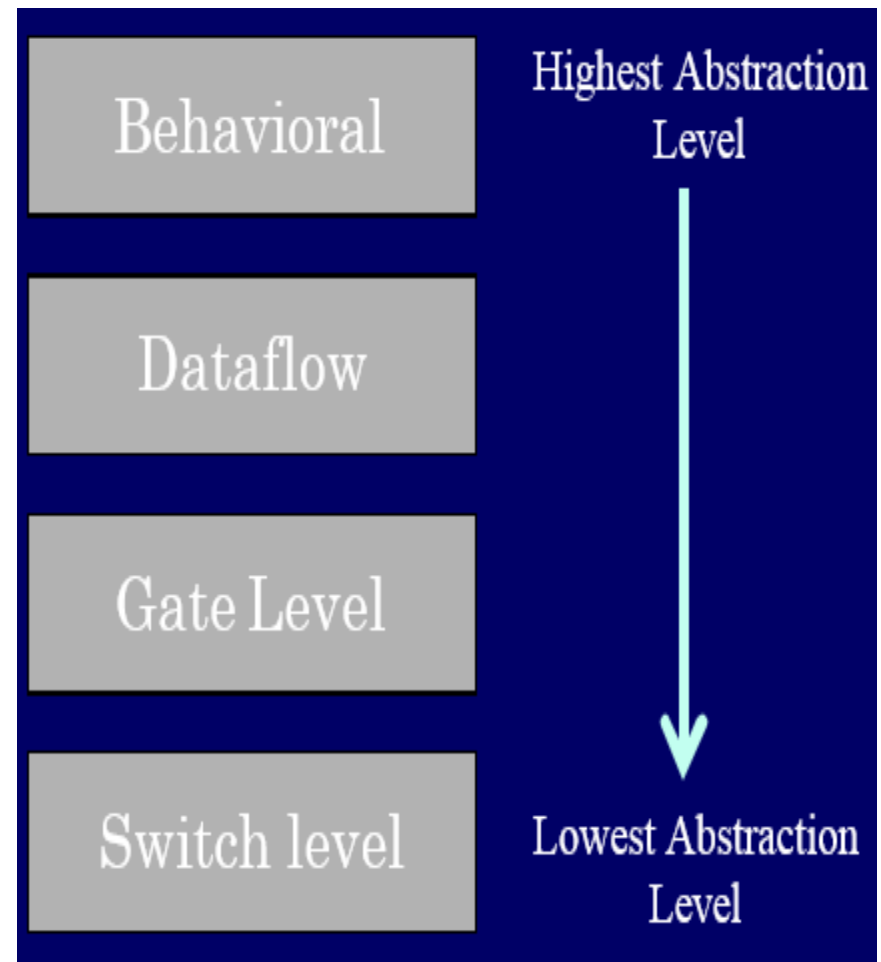
- S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Pearson, 2nd Ed, 2003.

VLSI Design Flow



Modeling Techniques

- Behavioral Modeling
- Data-flow Modeling
- Gate level Modeling
- Switch level Modeling



Switch-Level Modeling

//CMOS Inverter

```
module not_gate (x, f) ;
```

```
  input x ;
```

```
  output f ;
```

```
//internal declaration
```

```
supply1 vdd ;
```

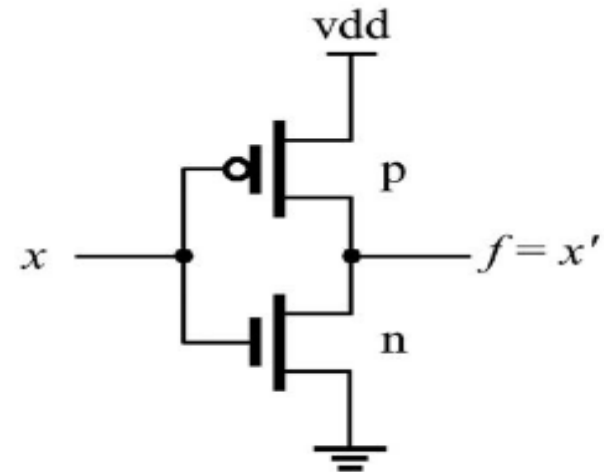
```
supply0 gnd ;
```

```
//NOT gate body
```

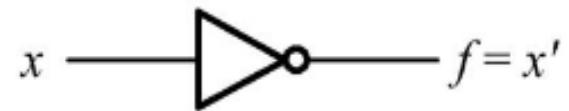
```
pmos p1 (f, vdd, x) ;
```

```
nmos n1 (f, gnd, x) ;
```

```
endmodule
```



(a) Circuit



(b) Logic symbol

Gate-Level Modeling

→ Need to model the gate's:

- Function
- Delay

→ Verilog provides built-in gate-level primitives:

and	nand	logical AND/NAND
or	nor	logical OR/NOR
xor	xnor	logical XOR/XNOR
buf	not	buffer/Inverter
bufif0	notif0	Tristate with low enable
bufif1	notif1	Tristate with High enable

Primitive_name #delay instance_name (out, in_1, in_2, ...) ;

and #1 a1 (out, in_1, in_2) ;

```
module simple_ckt (x, y, A, B, C) ;
```

```
  input A, B, C ;
```

```
  output x, y ;
```

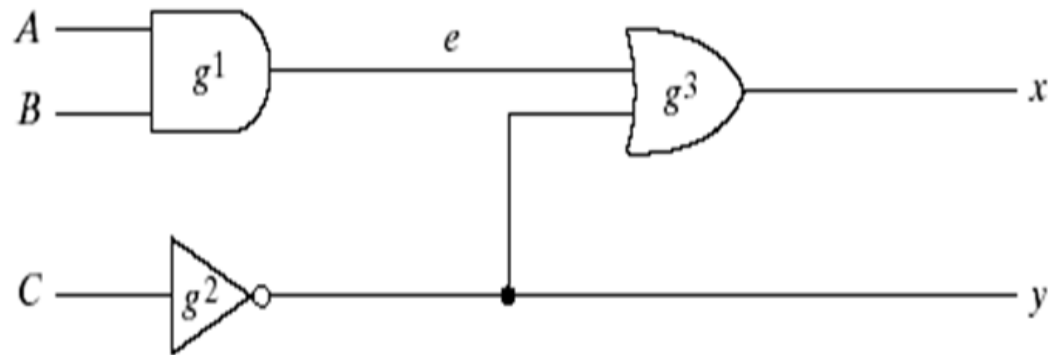
```
  wire e ;
```

```
  and g1 (e, A, B) ;
```

```
  not g2 (y, C) ;
```

```
  or g3 (x, e, y) ;
```

```
endmodule
```



//Gate-level description of 4-to-1- line multiplexer

```
module mux4x1 (I0, I1, I2, I3, s1, s0, y);
```

```
  input I0, I1, I2, I3, s1, s0;
```

```
  output y;
```

```
  //intermediate connections
```

```
  wire NOT_s1, NOT_s0 ;
```

```
  wire y0, y1, y2, y3 ;
```

```
  not n0 (NOT_s0, s0) ;
```

```
  not n1 (NOT_s1, s1) ;
```

```
  and a0 (y0, I0, NOT_s1, NOT_s0) ;
```

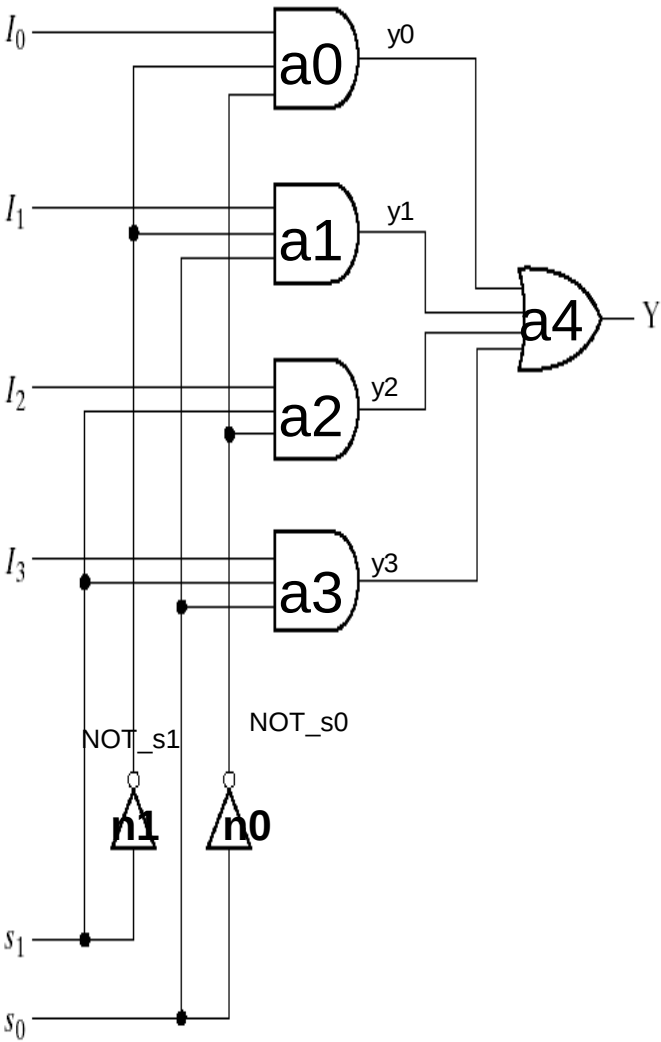
```
  and a1 (y1, I1, NOT_s1,      s0) ;
```

```
  and a2 (y2, I2,      s1, NOT_s0) ;
```

```
  and a3 (y3, I3,      s1,      s0) ;
```

```
  or a4 (y, y3, y2, y1, y0) ;
```

```
endmodule
```

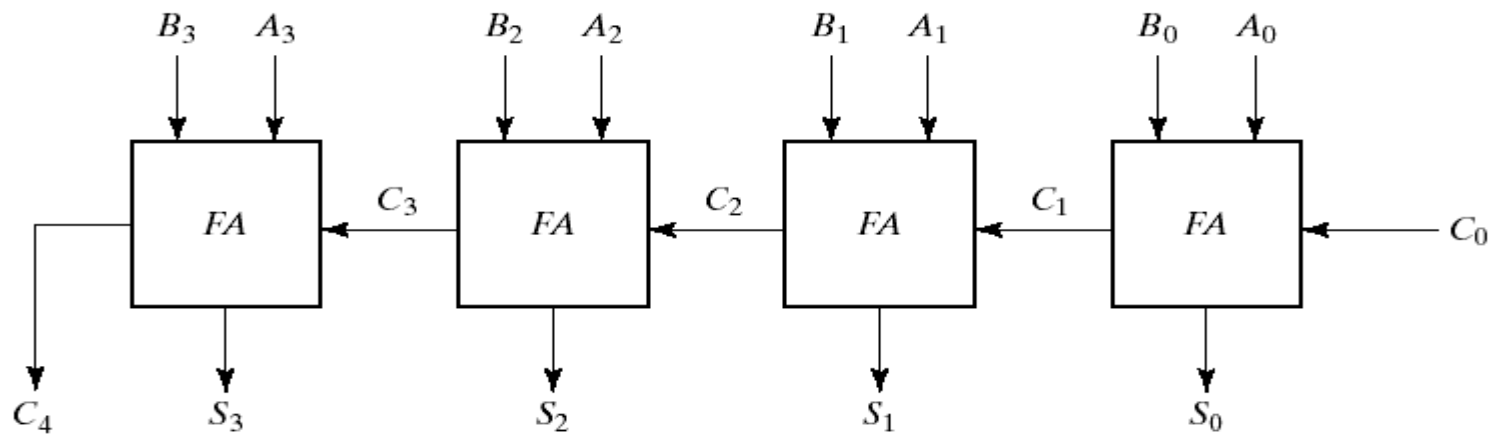
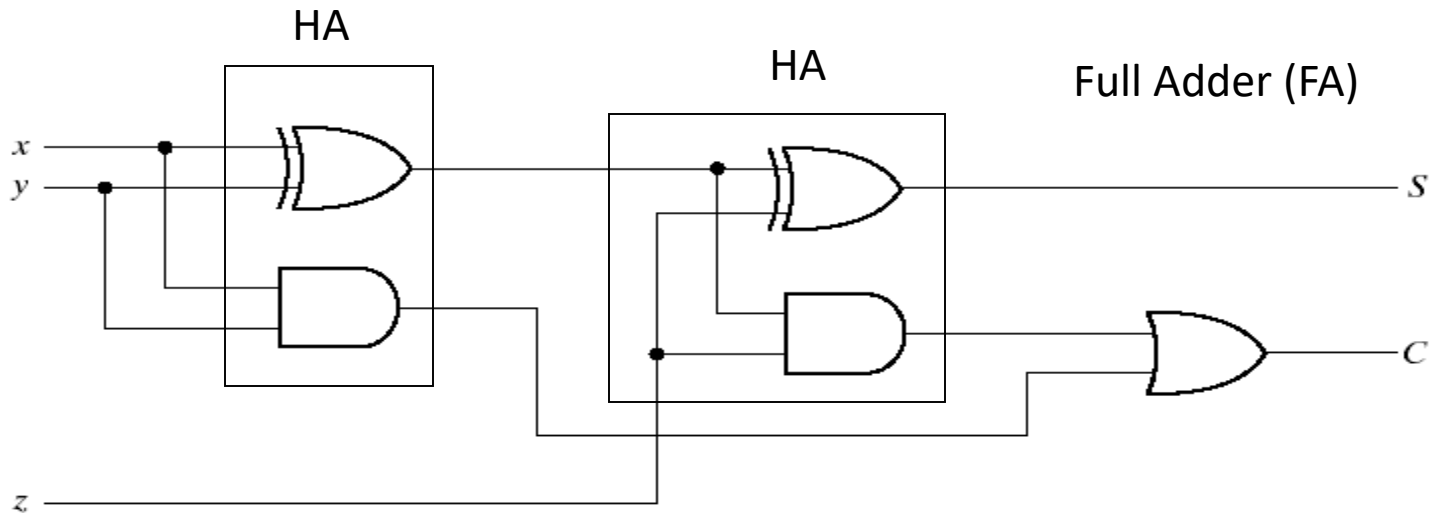


s ₁	s ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃

(b) Function table

Gate-level/Structural Modeling

4 bit Full Adder – Half Adder



4 bit Full Adder

//Gate-level hierarchical description of 4-bit adder

// Description of half adder

```
module halfadder (S, C, x, y);
```

```
  input x, y;
```

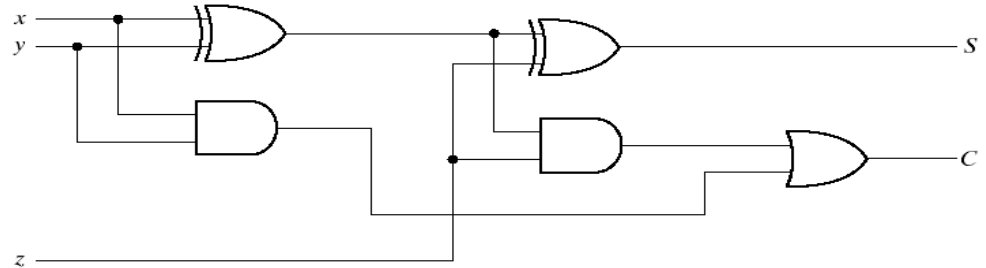
```
  output S, C;
```

```
  //Instantiate primitive gates
```

```
  xor (S, x, y);
```

```
  and (C, x, y);
```

```
endmodule
```



//Description of full adder

```
module fulladder (S, C, x, y, z);
```

```
  input x, y, z;
```

```
  output S, C;
```

```
  wire S1, D1, D2; //Outputs of first XOR and two AND gates
```

```
  //Instantiate the halfadder
```

```
  halfadder HA1 (S1, D1, x, y),
```

```
  halfadder HA2 (S, D2, S1, z);
```

```
  or      g1  (C, D2, D1);
```

```
endmodule
```

4 bit Full Adder

```
module four_bit_adder (S, C4, A, B, C0);
```

```
    input [3:0] A, B;
```

```
    input C0;
```

```
    output [3:0] S;
```

```
    output C4;
```

```
    wire C1,C2,C3; //Intermediate carries
```

```
    //Instantiate the full adder cells
```

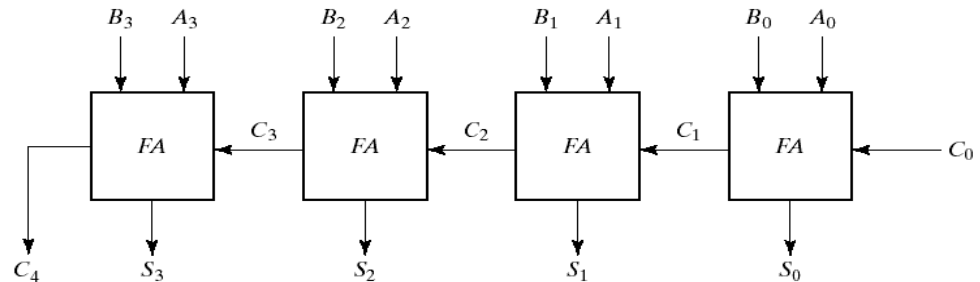
```
    fulladder FA0 (S[0],C1,A[0],B[0],C0),
```

```
                FA1 (S[1],C2,A[1],B[1],C1),
```

```
                FA2 (S[2],C3,A[2],B[2],C2),
```

```
                FA3 (S[3],C4,A[3],B[3],C3);
```

```
endmodule
```



Data-flow Modeling

- Dataflow modeling uses *continuous* assignments and the keyword “***assign***”.
- Syntax:
assign <drive_strength> #<delay> <list_of_assignments>;
- Example:
wire out ;
assign out = var1 & var2 ;
- The assignment **expression is evaluated as soon as one of the right-hand-side operands changes and the value is assigned to the left-hand-side net.**
- ***LHS*** must be net & cannot be reg. ***RHS*** can be net/reg.

//Data-flow description of 4-to-1- line multiplexer

```
module mux4x1 (I0, I1, I2, I3, s1, s0, y);
```

```
  input I0, I1, I2, I3, s1, s0;
```

```
  output y;
```

```
  assign y = (~s1 & ~s0 & I0) |  
             (~s1 &  s0 & I1) |  
             ( s1 & ~s0 & I2) |  
             ( s1 &  s0 & I3);
```

```
endmodule
```

//Using conditional operator

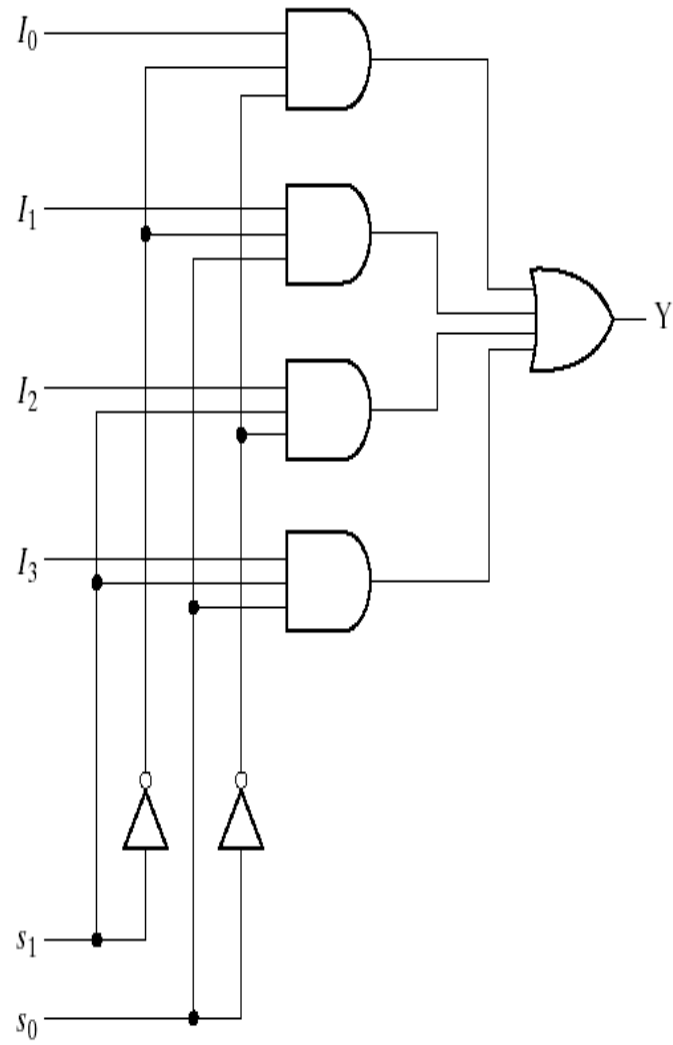
```
module mux2x1 (I0, I1, sel, y);
```

```
  input I0, I1, sel;
```

```
  output y;
```

```
  assign y = sel ? I1 : I0 ;
```

```
endmodule
```

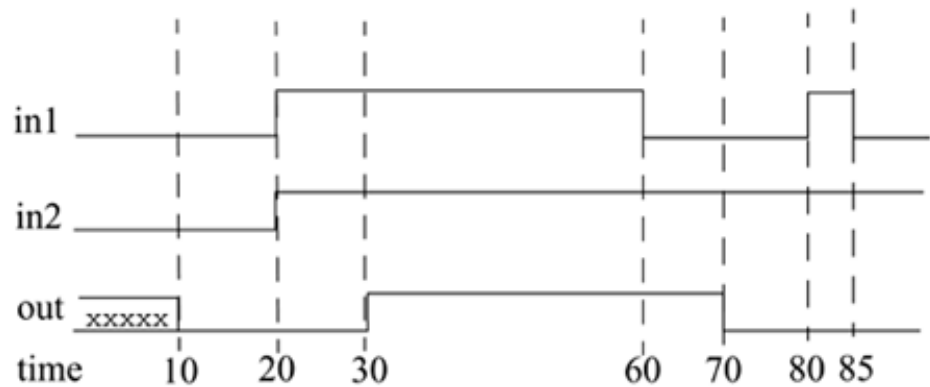


s ₁	s ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃

(b) Function table

Delays

- **assign #10 out = in1 & in2; // Delay in a continuous assign**



- Delays are non-synthesizable – used in test bench

Operator Type

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	*	multiply	two
	/	divide	two
	+	add	two
	-	subtract	two
	%	modulus	two
	**	power (exponent)	two
Logical	!	logical negation	one
	&&	logical and	two
		logical or	two
Relational	>	greater than	two
	<	less than	two
	>=	greater than or equal	two
	<=	less than or equal	two
Equality	==	equality	two
	!=	inequality	two

Operator Types

Bitwise	~	bitwise negation	one
	&	bitwise and	two
		bitwise or	two
	^	bitwise xor	two
	^~ or ~^	bitwise xnor	two
Reduction	&	reduction and	one
	~&	reduction nand	one
		reduction or	one
	~	reduction nor	one
	^	reduction xor	one
	^~ or ~^	reduction xnor	one
Shift	>>	Right shift	Two
	<<	Left shift	Two
	>>>	Arithmetic right shift	Two
	<<<	Arithmetic left shift	Two
Concatenation	{ }	Concatenation	Any number
Replication	{ { } }	Replication	Any number
Conditional	?:	Conditional	Three

Thank You